

Name: Manahil Habib

Reg no: 2023-BSE-042

Section: 5B

Lab 13

Task 0 Lab Setup (Codespace & GH CLI)

All actions below should be executed inside the Codespace shell.

Create Codespace & connect:

create or open codespace via GH CLI (example)

gh repo create CC_<YourName>_<YourRollNumber>/Lab13 --public

```
C:\Users\HABIB TARIQ\Lab12>gh repo create manahilhabib031/Lab13 --public
✓ Created repository manahilhabib031/Lab13 on github.com
https://github.com/manahilhabib031/Lab13
```

```
C:\Users\HABIB TARIQ\Lab12>
```

gh codespace create --repo <user_name>/Lab13

```
C:\Users\HABIB TARIQ\Lab12\Lab13>gh codespace create --repo manahilhabib031/Lab13
```

```
✓ Codespaces usage for this repository is paid for by manahilhabib031
? Choose Machine Type: 2 cores, 8 GB RAM, 32 GB storage
expert-garbanzo-wrq7qxqx66r43g57x
```

```
C:\Users\HABIB TARIQ\Lab12\Lab13>
```

gh codespace list

gh codespace ssh -c <your_codespace_name>

```
C:\Users\HABIB TARIQ\Lab12\Lab13>gh codespace ssh -c expert-garbanzo-wrq7qxqx66r43g57x
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)
```

```
* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

```
@manahilhabib031 → /workspaces/Lab13 (main) $
```

Task 1 — Create IAM Group and Output Details

In this task, you will create an IAM group named "developers" and output its details.

1. Create the initial project structure:

```
mkdir -p ~/Lab13
```

```
cd ~/Lab13
```

```
@manahilhabib031 → /workspaces/Lab13 (main) $ mkdir -p ~/Lab13
@manahilhabib031 → /workspaces/Lab13 (main) $ cd ~/Lab13
@manahilhabib031 → ~/Lab13 $ |
```

2. Create the main Terraform file:

```
touch main.tf
```

```
@manahilhabib031 → ~/Lab13 $ touch main.tf
@manahilhabib031 → ~/Lab13 $ ls
main.tf
@manahilhabib031 → ~/Lab13 $
```

3. Create main.tf with AWS provider configuration:

```
provider "aws" {
```

```
    shared_config_files    = ["~/.aws/config"]
```

```
    shared_credentials_files = ["~/.aws/credentials"]
```

```
}
```

```
resource "aws_iam_group" "developers" {
```

```
    name = "developers"
```

```
    path = "/groups/"
```

```
}
```

```
output "group_details" {
```

```
    value = {
```

```
        group_name = aws_iam_group.developers.name
```

```
        group_arn  = aws_iam_group.developers.arn
```

```
        unique_id  = aws_iam_group.developers.unique_id
```

```
    }
```

```
}
```

```

GNU nano 7.2                                main.tf *
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
    group_arn  = aws_iam_group.developers.arn
    unique_id  = aws_iam_group.developers.unique_id
  }
}
|

```

4. Initialize Terraform:

terraform init

```

@manahilhabib031 → ~/Lab13 $ terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 → ~/Lab13 $ |

```

5. Apply the configuration:

terraform apply -auto-approve

```

@manahilhabib031 → ~/Lab13 $ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_iam_group.developers will be created
+ resource "aws_iam_group" "developers" {
+   arn      = (known after apply)
+   id       = (known after apply)
+   name     = "developers"
+   path     = "/groups/"
+   unique_id = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ group_details = {
+   group_arn = (known after apply)
+   group_name = "developers"
+   unique_id = (known after apply)
}
aws_iam_group.developers: Creating...
aws_iam_group.developers: Creation complete after 1s [id=developers]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
@manahilhabib031 → ~/Lab13 $ |

```

6. Display the output:

terraform output

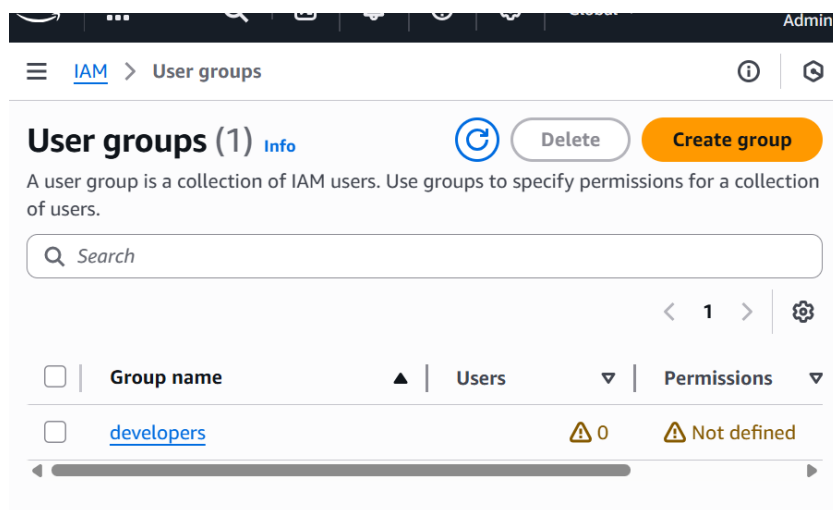
```

@manahilhabib031 → ~/Lab13 $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
@manahilhabib031 → ~/Lab13 $ |

```

7. Verify the group in AWS Console:

- Navigate to IAM → Groups in AWS Console



Task 2 — Create IAM User with Group Membership

In this task, you will create an IAM user named "loadbalancer" and add it to the developers group.

1. Update main.tf to add the IAM user resource:

```
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
  
resource "aws_iam_group" "developers" {  
  name = "developers"  
  path = "/groups/"  
}  
  
output "group_details" {  
  value = {  
    group_name = aws_iam_group.developers.name  
    group_arn  = aws_iam_group.developers.arn  
    unique_id  = aws_iam_group.developers.unique_id  
  }  
}  
  
resource "aws_iam_user" "lb" {  
  name = "loadbalancer"  
  path = "/users/"  
  force_destroy = true  
  tags = {  
    DisplayName = "Load Balancer"  
  }  
}  
  
resource "aws_iam_user_group_membership" "lb_membership" {
```

```

user = aws_iam_user.lb.name

groups = [
    aws_iam_group.developers.name
]
}

output "user_details" {
    value = {
        user_name = aws_iam_user.lb.name
        user_arn  = aws_iam_user.lb.arn
        unique_id = aws_iam_user.lb.unique_id
    }
}

```



The screenshot shows a terminal window with the title 'GNU nano 7.2 main.tf *'. The content is a Terraform configuration file with the following structure:

```

provider "aws" {
    shared_config_files = ["~/.aws/config"]
    shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
    name = "developers"
    path = "/groups/"
}

resource "aws_iam_user" "lb" {
    name          = "loadbalancer"
    path          = "/users/"
    force_destroy = true
    tags = {
        DisplayName = "Load Balancer"
    }
}

resource "aws_iam_user_group_membership" "lb_membership" {
    user = aws_iam_user.lb.name
    groups = [
        aws_iam_group.developers.name
    ]
}

output "user_details" {
    value = {
        user_name = aws_iam_user.lb.name
        user_arn  = aws_iam_user.lb.arn
        unique_id = aws_iam_user.lb.unique_id
    }
}

output "group_details" {
    value = {

```

2. Apply the configuration:

```
terraform apply -auto-approve
```

```
@manahilhabib031 → ~/Lab13 $ terraform apply -auto-approve
aws_iam_group.developers: Refreshing state... [id=developers]

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_iam_user.lb will be created
+ resource "aws_iam_user" "lb" {
  + arn          = (known after apply)
  + force_destroy = true
  + id           = (known after apply)
  + name         = "loadbalancer"
  + path         = "/users/"
  + tags         = {
    + "DisplayName" = "Load Balancer"
  }
  + tags_all     = {
    + "DisplayName" = "Load Balancer"
  }
  + unique_id    = (known after apply)
}

# aws_iam_user_group_membership.lb_membership will be created
+ resource "aws_iam_user_group_membership" "lb_membership" {
  + groups = [
    + "developers",
  ]
}
```

```
Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ user_details = {
  + unique_id = (known after apply)
  + user_arn  = (known after apply)
  + user_name = "loadbalancer"
}
aws_iam_user.lb: Creating...
aws_iam_user.lb: Creation complete after 1s [id=loadbalancer]
aws_iam_user_group_membership.lb_membership: Creating...
aws_iam_user_group_membership.lb_membership: Creation complete after 0s [id=
=terraform-20260111040806241800000001]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
user_details = {
  "unique_id" = "AIDARMIC7HAZLTUMFDUJJ"
  "user_arn" = "arn:aws:iam::095032391730:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@manahilhabib031 → ~/Lab13 $ |
```

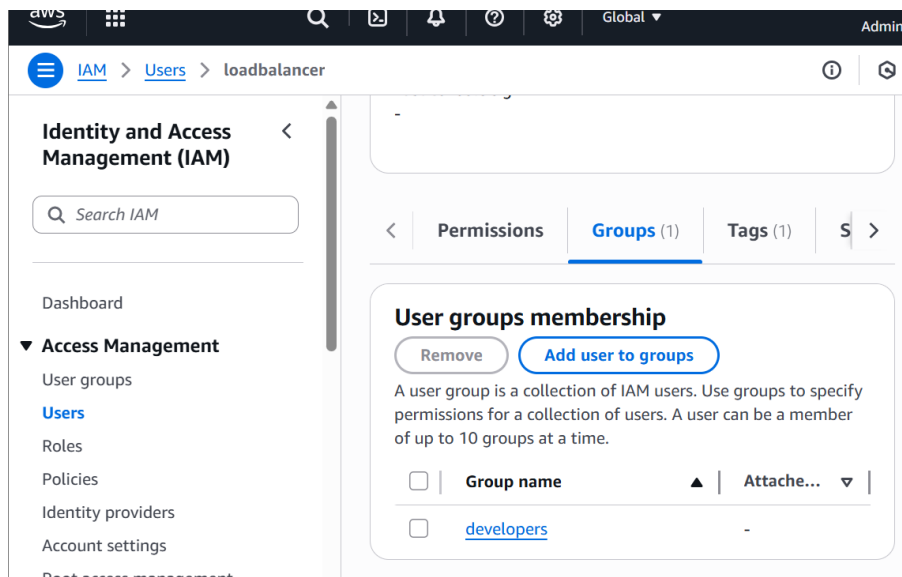
3. Display the outputs:

terraform output

```
@manahilhabib031 → ~/Lab13 $ terraform output
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
user_details = {
  "unique_id" = "AIDARMIC7HAZLTUMFDUJJ"
  "user_arn" = "arn:aws:iam::095032391730:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@manahilhabib031 → ~/Lab13 $ |
```

4. Verify the user in AWS Console:

- Navigate to IAM → Users in AWS Console
- Click on "loadbalancer" user
- Check the "Groups" tab



Task 3 — Attach Policies to IAM Group

In this task, you will attach AWS managed policies (AmazonEC2FullAccess and IAMUserChangePassword) to the developers group.

1. Update main.tf to add policy attachments:

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

resource "aws_iam_group" "developers" {
  name = "developers"
  path = "/groups/"
}

output "group_details" {
  value = {
    group_name = aws_iam_group.developers.name
  }
}
```



```
    group_arn = aws_iam_group.developers.arn
    unique_id = aws_iam_group.developers.unique_id
  }
}
```

```
resource "aws_iam_user" "lb" {
  name = "loadbalancer"
  path = "/users/"
  force_destroy = true
  tags = {
    DisplayName = "Load Balancer"
  }
}
```

```
resource "aws_iam_user_group_membership" "lb_membership" {
  user = aws_iam_user.lb.name
  groups = [
    aws_iam_group.developers.name
  ]
}
```

```
output "user_details" {
  value = {
    user_name = aws_iam_user.lb.name
    user_arn = aws_iam_user.lb.arn
    unique_id = aws_iam_user.lb.unique_id
  }
}
```

```
resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
  group = aws_iam_group.developers.name
}
```

```

    policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

resource "aws_iam_group_policy_attachment" "change_password" {
    group = aws_iam_group.developers.name
    policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

```

```

}
resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
    group      = aws_iam_group.developers.name
    policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

resource "aws_iam_group_policy_attachment" "change_password" {
    group      = aws_iam_group.developers.name
    policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

```

2. Apply the configuration:

terraform apply -auto-approve

```

@manahilhabib031 → ~/Lab13 $ terraform apply -auto-approve
aws_iam_group.developers: Refreshing state... [id=developers]
aws_iam_user.lb: Refreshing state... [id=loadbalancer]
aws_iam_user_group_membership.lb_membership: Refreshing state... [id=terraform-202601110408062418000000001]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# aws_iam_group_policy_attachment.change_password will be created
+ resource "aws_iam_group_policy_attachment" "change_password" {
  + group      = "developers"
  + id         = (known after apply)
  + policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}

# aws_iam_group_policy_attachment.developer_ec2_fullaccess will be created
+ resource "aws_iam_group_policy_attachment" "developer_ec2_fullaccess" {
  + group      = "developers"
  + id         = (known after apply)
  + policy_arn = "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
}

Plan: 2 to add, 0 to change, 0 to destroy.
aws_iam_group_policy_attachment.change_password: Creating...

```

```
Plan: 2 to add, 0 to change, 0 to destroy.
aws_iam_group_policy_attachment.change_password: Creating...
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Creating...
aws_iam_group_policy_attachment.change_password: Creation complete after 1s
-202601110414256631000000001]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Creation complete
evelopers-202601110414256738000000002]
```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
user_details = {
  "unique_id" = "AIDARMIC7HAZLTUMFDUJJ"
  "user_arn" = "arn:aws:iam::095032391730:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@manahilhabib031 → ~/Lab13 $
```

3. Verify policies in AWS Console:

- Navigate to IAM → Groups → developers
- Click on "Permissions" tab

The screenshot shows the AWS IAM console interface. The breadcrumb navigation is IAM > User groups > developers. The 'Permissions' tab is selected, showing 'Permissions policies (2)'. There are buttons for 'Simulate', 'Remove', and 'Add permissions'. A message states 'You can attach up to 10 managed policies.' Below this is a 'Filter by Type' section with a search bar and a dropdown set to 'All types'. A table lists the attached policies:

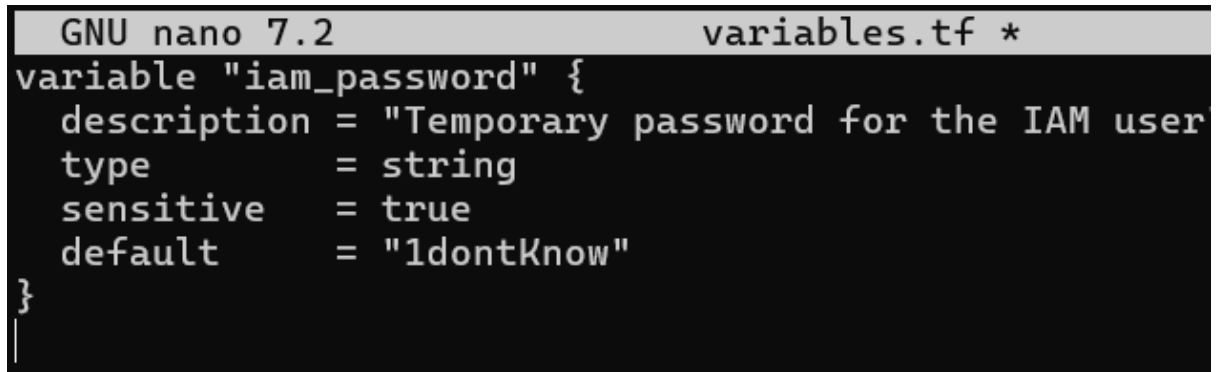
☐	Policy name	Type	Attached entities
☐	AmazonEC2FullAc...	AWS managed	1
☐	IAMUserChangePa...	AWS managed	3

Task 4 — Create Login Profile for IAM User

In this task, you will create a login profile for the loadbalancer user using a bash script and null_resource provisioner.

1. Create variables.tf file:

```
variable "iam_password" {  
    description = "Temporary password for the IAM user"  
    type        = string  
    sensitive   = true  
    default     = "1dontKnow"  
}
```



```
GNU nano 7.2 variables.tf *  
variable "iam_password" {  
    description = "Temporary password for the IAM user"  
    type        = string  
    sensitive   = true  
    default     = "1dontKnow"  
}
```

2. Create the bash script create-login-profile.sh:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
USERNAME="$1"
```

```
PASSWORD="$2"
```

```
# Check if login profile already exists
```

```
if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>&1; then
```

```
    echo "Login profile already exists for $USERNAME. Skipping."
```

```
else
```

```
    echo "Creating login profile for $USERNAME"
```

```
    aws iam create-login-profile \
```

```
        --user-name "$USERNAME" \
```

```
        --password "$PASSWORD" \
```

```
        --password-reset-required
```

```
fi
```

```

GNU nano 7.2      create-login-profile.sh *
#!/usr/bin/env bash
set -euo pipefail

USERNAME="$1"
PASSWORD="$2"

# Check if login profile already exists
if aws iam get-login-profile --user-name "$USERNAME" >/dev/null 2>>
    echo "Login profile already exists for $USERNAME. Skipping."
else
    echo "Creating login profile for $USERNAME"
    aws iam create-login-profile \
        --user-name "$USERNAME" \
        --password "$PASSWORD" \
        --password-reset-required
fi
|

```

3. Make the script executable:

```
chmod +x create-login-profile.sh
```

```

@manahilhabib031 → /workspaces/Lab13 (main) $ nano variables.tf
@manahilhabib031 → /workspaces/Lab13 (main) $ nano create-login-profile.sh
@manahilhabib031 → /workspaces/Lab13 (main) $ chmod +x create-login-profile.sh
@manahilhabib031 → /workspaces/Lab13 (main) $ |

```

4. Update main.tf to add the null_resource provisioner:

Add this resource after the user creation:

```

resource "null_resource" "create_login_profile" {

  triggers = {

    password_hash = sha256(var.iam_password)

    user      = aws_iam_user.lb.name
  }
}

```

```
depends_on = [aws_iam_user.lb]
```

```

provisioner "local-exec" {

  command = "${path.module}/create-login-profile.sh ${aws_iam_user.lb.name}
'${var.iam_password}'"
}

```

```
}
```

```
GNU nano 7.2 main.tf *
resource "null_resource" "create_login_profile" {
  triggers = {
    password_hash = sha256(var.iam_password)
    user          = aws_iam_user.lb.name
  }

  depends_on = [aws_iam_user.lb]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${aws_iam_us
  }
}
```

5. Apply the configuration with a custom password:

```
terraform apply -auto-approve -var="iam_password=MySecurePass123!"
```

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform apply -auto
-approve -var="iam_password=MySecurePass123!"
aws_iam_user.lb: Refreshing state... [id=loadbalancer]
aws_iam_group.developers: Refreshing state... [id=developers]
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create
- ~ update in-place

Terraform will perform the following actions:

```
# aws_iam_group_policy_attachment.change_password will be created
+ resource "aws_iam_group_policy_attachment" "change_password" {
  + group      = "developers"
  + id         = (known after apply)
  + policy_arn = "arn:aws:iam::aws:policy/IAMUserChangePassword"
}
```

```
# aws_iam_group_policy_attachment.developer_ec2_fullaccess will be created
```

```
+ resource "aws_iam_group_policy_attachment" "developer_ec2_fulla
ccess" {
  + group      = "developers"
  + id         = (known after apply)
```

```
1133382489872148841]
```

Apply complete! Resources: 4 added, 1 changed, 0 destroyed.

Outputs:

```
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
user_details = {
  "unique_id" = "AIDARMIC7HAZLTUMFDUJJ"
  "user_arn" = "arn:aws:iam::095032391730:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

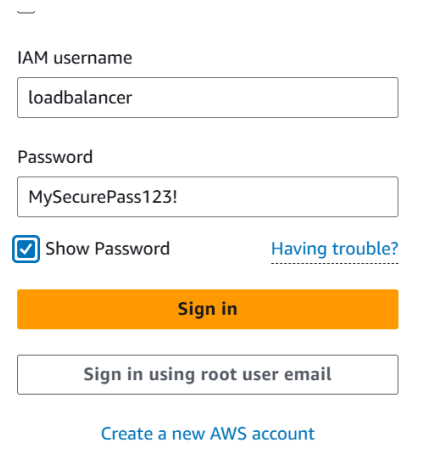
6. Verify login profile creation:

aws iam get-login-profile --user-name loadbalancer

```
@manahilhabib031 → /workspaces/Lab13 (main) $ aws iam get-login-profile --user-name loadbalancer
{
  "LoginProfile": {
    "UserName": "loadbalancer",
    "CreateDate": "2026-01-11T05:14:36+00:00",
    "PasswordResetRequired": true
  }
}
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

7. Test login in AWS Console:

- Open AWS Console login page
- Sign in as IAM user with username "loadbalancer" and the password you set
- You should be prompted to change password



IAM username

Password

☒ Show Password [Having trouble?](#)

[Sign in](#)

[Sign in using root user email](#)

[Create a new AWS account](#)

Password reset ⓘ

Your account (095032391730) password has expired or requires a reset.

To continue, please verify your old and set a new password for **loadbalancer** (not you?).

Old Password

☐ Show Password

New Password

Confirm New Password

☐ Show Password

Confirm Password Change

Task 5 — Generate Access Keys for IAM User

In this task, you will create access keys for the loadbalancer user and view them in terraform state.

1. Update main.tf to add access key resource and outputs:

Add these resources:

```
resource "aws_iam_access_key" "lb_access_key" {
  user = aws_iam_user.lb.name
}

output "access_key_id" {
  value = aws_iam_access_key.lb_access_key.id
}

output "access_key_secret" {
  value  = aws_iam_access_key.lb_access_key.secret
  sensitive = true
}
```



```
# -----
# Task 5: IAM Access Keys
# -----
resource "aws_iam_access_key" "lb_access_key" {
  user = aws_iam_user.lb.name
}

output "access_key_id" {
  value = aws_iam_access_key.lb_access_key.id
}

output "access_key_secret" {
  value      = aws_iam_access_key.lb_access_key.secret
  sensitive = true
}
|
```

2. Apply the configuration:

terraform apply -auto-approve -var="iam_password=MySecurePass123!"

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform apply -auto-approve -var="iam_password=MySecurePass123!"
aws_iam_group.developers: Refreshing state... [id=developers]
aws_iam_user.lb: Refreshing state... [id=loadbalancer]
aws_iam_group_policy_attachment.change_password: Refreshing state... [id=developers-202601110514289186000000002]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Refreshing state... [id=developers-202601110514287011000000001]
aws_iam_user_group_membership.lb_membership: Refreshing state... [id=terraform-202601110514289592000000003]
null_resource.create_login_profile: Refreshing state... [id=1155562469672148841]

Terraform used the selected providers to generate the following
execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_iam_access_key.lb_access_key will be created
+ resource "aws_iam_access_key" "lb_access_key" {
  + create_date      = (known after apply)
  + encrypted_secret = (known after apply)
  + encrypted_ses_smtp_password_v4 = (known after apply)
  + id               = (known after apply)
  + key_fingerprint  = (known after apply)
  + secret           = (sensitive value)
  + ses_smtp_password_v4 = (sensitive value)
  + status           = "Active"
  + user             = "loadbalancer"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ access_key_id      = (known after apply)
+ access_key_secret = (sensitive value)
aws_iam_access_key.lb_access_key: Creating...
aws_iam_access_key.lb_access_key: Creation complete after 1s [id=AKIARMIC7HAZHCRXY2BU]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

3. Display outputs:

terraform output

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform output
access_key_id = "AKIARMIC7HAZHCRXY2BU"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
user_details = {
  "unique_id" = "AIDARMIC7HAZLTUMFDUJJ"
  "user_arn" = "arn:aws:iam::095032391730:user/users/loadbalancer"
  "user_name" = "loadbalancer"
}
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

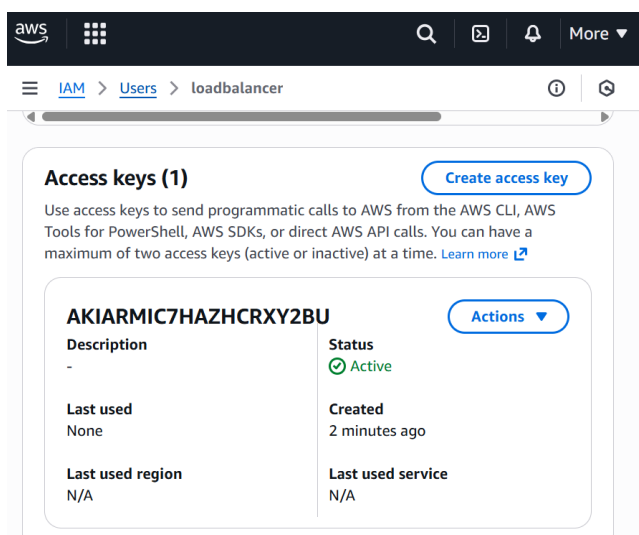
4. View the secret in terraform state:

```
cat terraform.tfstate | grep -A 10 "access_key_secret"
```

```
@manahilhabib031 → /workspaces/Lab13 (main) $ cat terraform.tfstate | grep -A 10 "access_key_secret"
"access_key_secret": {
  "value": "Zx3LzIsxgCbLwvA3V3JKTMZ2D2+UleokDyY/2pCk",
  "type": "string",
  "sensitive": true
},
"group_details": {
  "value": {
    "group_arn": "arn:aws:iam::095032391730:group/groups/developers",
    "group_name": "developers",
    "unique_id": "AGPARMIC7HAZH3R4ACGUN"
  },
}
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

5. Verify access key in AWS Console:

- Navigate to IAM → Users → loadbalancer → Security credentials

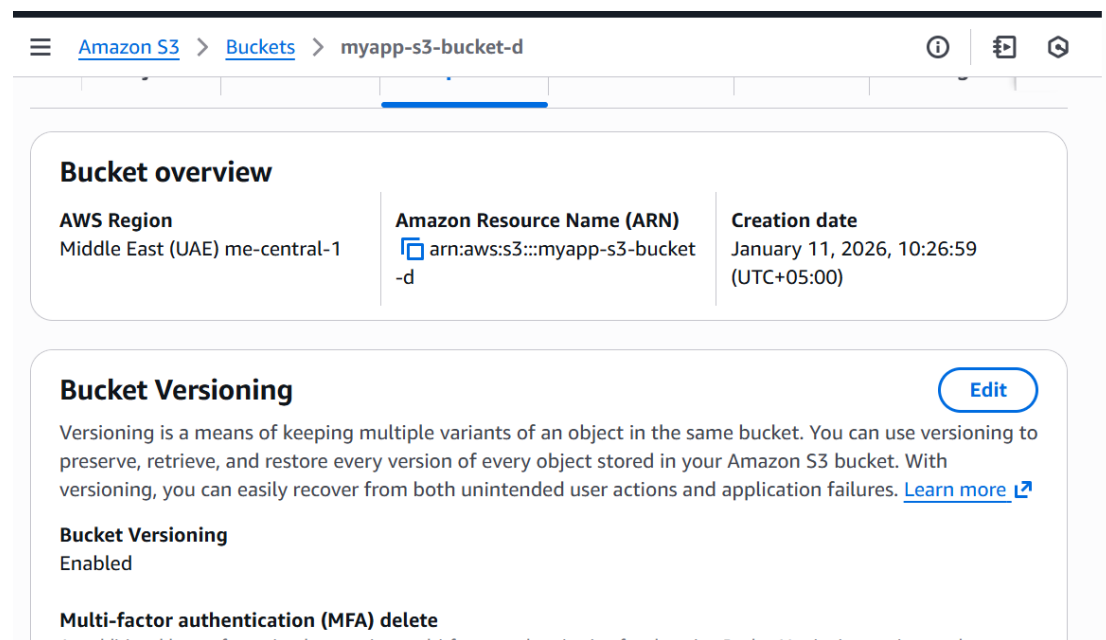
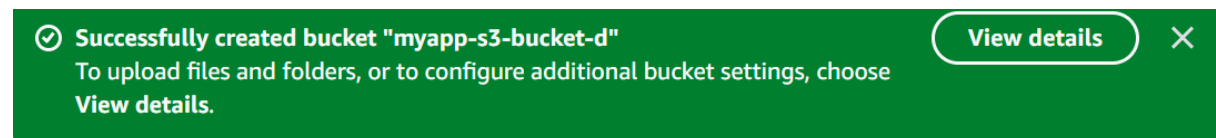


Task 6 — Implement Terraform Remote State with S3

In this task, you will configure Terraform to use S3 backend for remote state storage.

1. Create S3 bucket in AWS Console:

- Navigate to S3 in AWS Console
- Click "Create bucket"
- Bucket name: myapp-s3-bucket-demo (use a unique name if this is taken)
- Enable versioning
- Keep other settings as default
- Click "Create bucket"



2. Update main.tf to add S3 backend configuration:

Add this at the beginning of main.tf (before the provider block):

```
terraform {  
  backend "s3" {  
    bucket = "myapp-s3-bucket-demo"  
    key    = "myapp/terraform.tfstate"  
    region = "me-central-1"  
    encrypt = true  
    use_lockfile = true  
  }  
}
```

```
}
```

```
GNU nano 7.2      main.tf
terraform {
  backend "s3" {
    bucket = "myapp-s3-bucket-d"
    key    = "myapp/terraform.tfstate"
    region = "me-central-1"
    encrypt = true
  }
}
```

3. Reinitialize Terraform with the backend:

terraform init -migrate-state

- Type yes when prompted to migrate state

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform init
-migrate-state

Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "
local" backend to the
  newly configured "s3" backend. No existing state was found
in the newly
  configured "s3" backend. Do you want to copy this state to
the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty
state.

Enter a value: yes

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
- Reusing previous version of hashicorp/null from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/null v3.2.4
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

4. Apply the configuration:

terraform apply -auto-approve -var="iam_password=MySecurePass123!"

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform apply
-auto-approve -var="iam_password=MySecurePass123!"
aws_iam_group.developers: Refreshing state... [id=developers]
aws_iam_user.lb: Refreshing state... [id=loadbalancer]
null_resource.create_login_profile: Refreshing state... [id=1
155562469672148841]
aws_iam_access_key.lb_access_key: Refreshing state... [id=AKI
ARMIC7HAZHCRXY2BU]
aws_iam_group_policy_attachment.change_password: Refreshing s
tate... [id=developers-20260111051428918600000002]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Ref
reshing state... [id=developers-20260111051428701100000001]
aws_iam_user_group_membership.lb_membership: Refreshing state
... [id=terraform-20260111051428959200000003]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your
configuration and found no differences, so no changes are
needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

access_key_id = "AKIARMIC7HAZHCRXY2BU"
access_key_secret = <sensitive>
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/devel
opers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZH3R4ACGUN"
}
user_details = {
  "unique_id" = "AIDARMIC7HAZLTUMFDUJJ"
  "user_arn" = "arn:aws:iam::095032391730:user/users/loadbala
ncer"
  "user_name" = "loadbalancer"
}
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

5. Verify state file in S3:

- Navigate to S3 → myapp-s3-bucket-demo → myapp/
- You should see terraform.tfstate file

The screenshot shows the Amazon S3 console interface. The breadcrumb navigation indicates the path: Amazon S3 > Buckets > myapp-s3-bucket-d > myapp/. The 'Objects (1)' section displays a table with one object:

Name	Type	Last modified	Size	Storage class
terraform.tfstate	tfstate	January 11, 2026, 10:42:31 (UTC+05:00)	6.2 KB	Standard

6. Check local state file:

ls -la terraform.tfstate*

```
@manahilhabib031 → /workspaces/Lab13 (main) $ ls -la terraform.tfstate*
-rw-rw-rw- 1 codespace codespace 0 Jan 11 05:42 terraform.tfstate
-rw-rw-rw- 1 codespace codespace 6339 Jan 11 05:42 terraform.tfstate.backup
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

7. Destroy resources and verify state change:

terraform destroy -auto-approve

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform destroy -auto-approve
aws_iam_group.developers: Refreshing state... [id=developers]
aws_iam_user.lb: Refreshing state... [id=loadbalancer]
aws_iam_access_key.lb_access_key: Refreshing state... [id=AKIARMIC7HAZHCRXY2BU]
null_resource.create_login_profile: Refreshing state... [id=1155562469672148841]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Refreshing state... [id=developers-20260111051428701100000001]
aws_iam_group_policy_attachment.change_password: Refreshing state... [id=developers-20260111051428918600000002]
aws_iam_user_group_membership.lb_membership: Refreshing state... [id=terraform-20260111051428959200000003]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# aws_iam_access_key.lb_access_key will be destroyed
- resource "aws_iam_access_key" "lb_access_key" {
  - create_date      = "2026-01-11T05:20:09Z" -> null
  - id              = "AKIARMIC7HAZHCRXY2BU" -> null
  - secret          = (sensitive value) -> null
  - ses_smtp_password_v4 = (sensitive value) -> null
  - status          = "Active" -> null
  - user            = "loadbalancer" -> null
}

# aws_iam_group.developers will be destroyed
- resource "aws_iam_group" "developers" {
  - arn = "arn:aws:iam::095032391730:group/groups/developers" -> null
```

```

- user_details = {
  - unique_id = "AIDARMIC7HAZLTUMFDUJJ"
  - user_arn = "arn:aws:iam::095032391730:user/users/loadbalancer"
  - user_name = "loadbalancer"
} -> null
null_resource.create_login_profile: Destroying... [id=1155562469672148841]
null_resource.create_login_profile: Destruction complete after 0s
aws_iam_group_policy_attachment.change_password: Destroying... [id=developers-202601110514289186000000002]
aws_iam_user_group_membership.lb_membership: Destroying... [id=terraform-202601110514289592000000003]
aws_iam_access_key.lb_access_key: Destroying... [id=AKIARMIC7HAZHCRXY2BU]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destroying... [id=developers-202601110514287011000000001]
aws_iam_user_group_membership.lb_membership: Destruction complete after 1s
aws_iam_access_key.lb_access_key: Destruction complete after 1s
aws_iam_user.lb: Destroying... [id=loadbalancer]
aws_iam_group_policy_attachment.developer_ec2_fullaccess: Destruction complete after 1s
aws_iam_group_policy_attachment.change_password: Destruction complete after 1s
aws_iam_group.developers: Destroying... [id=developers]
aws_iam_group.developers: Destruction complete after 0s
aws_iam_user.lb: Destruction complete after 3s

Destroy complete! Resources: 7 destroyed.
manahilhabib031 → /workspaces/Lab13 (main) $ |

```

8. Verify updated state in S3:

- Refresh S3 bucket view
- Check the terraform.tfstate file (it should show empty resources)

The screenshot shows the Amazon S3 console interface. The breadcrumb navigation indicates the path: Amazon S3 > ... > myapp/ > terraform.tfstate. The file 'terraform.tfstate' is selected, and its details are displayed. The 'Object overview' section shows the following information:

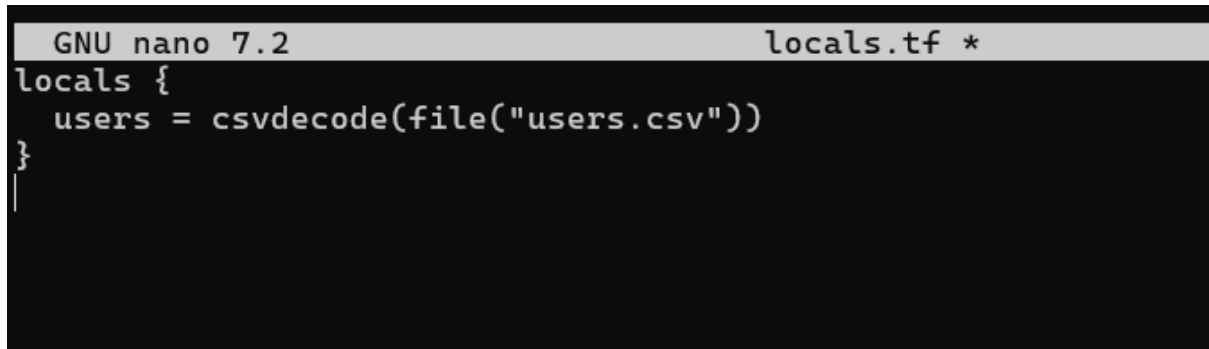
Property	Value
Owner	2a9e754f55125611d5950d8775a55397c9600a8ff4118f34101d28796c9e6c3c
AWS Region	Middle East (UAE) me-central-1
Last modified	15-Nov-2026 10:47:04
S3 URI	s3://myapp-s3-bucket-d/myapp/terraform.tfstate
Amazon Resource Name (ARN)	arn:aws:s3:::myapp-s3-bucket-d/myapp/terraform.tfstate
Entity tag (Etag)	5470b251057042...d0b4-375514

Task 7 — Create Multiple Users from CSV File

In this task, you will create multiple IAM users dynamically from a CSV file.

1. Create locals.tf file:

```
locals {  
  users = csvdecode(file("users.csv"))  
}
```



```
GNU nano 7.2 locals.tf *  
locals {  
  users = csvdecode(file("users.csv"))  
}
```

2. Create users.csv file:

user_name

Michaela

Dwight

Jim

Pam

Ryan

Andy

Robert

Stanley

Kevin

Angela

Oscar

Phyllis

Toby

Kelly

Darryl

Creed

Meredith

Erin

Gabe

Jan

David

Holly

Charles

Jo

Clark

Peter



```
GNU nano 7.2 users.csv *
user_name
Michael
Dwight
Jim
Pam
Ryan
Andy
Robert
Stanley
Kevin
Angela
Oscar
Phyllis
Toby
Kelly
Darryl
Creed
Meredith
Erin
Gabe
Jan
David
Holly
Charles
Jo
Clark
Peter
|
```

3. Update main.tf to create multiple users:

Replace the single user resources with:

Create multiple IAM users from CSV

```
resource "aws_iam_user" "users" {
  for_each = { for user in local.users : user.user_name => user }

  name      = each.value.user_name
  path      = "/users/"
  force_destroy = true
```

```
tags = {
  DisplayName = each.value.user_name
  CreatedBy   = "Terraform"
}
}

# Add all users to developers group
resource "aws_iam_user_group_membership" "users_membership" {
  for_each = aws_iam_user.users

  user = each.value.name
  groups = [
    aws_iam_group.developers.name
  ]
}

# Create login profiles for all users
resource "null_resource" "create_login_profiles" {
  for_each = aws_iam_user.users

  triggers = {
    password_hash = sha256(var.iam_password)
    user          = each.value.name
  }

  depends_on = [aws_iam_user.users]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${each.value.name} '${var.iam_password}'"
  }
}
```

```
# Create access keys for all users
```

```
resource "aws_iam_access_key" "users_access_keys" {
```

```
  for_each = aws_iam_user. users
```

```
  user = each.value.name
```

```
}
```

```
# Output all user details
```

```
output "all_users_details" {
```

```
  value = {
```

```
    for user_name, user in aws_iam_user.users : user_name => {
```

```
      user_arn      = user.arn
```

```
      user_unique_id = user.unique_id
```

```
      access_key_id = aws_iam_access_key.users_access_keys[user_name].id
```

```
    }
```

```
  }
```

```
}
```

```
# Output all access key secrets (sensitive)
```

```
output "all_access_key_secrets" {
```

```
  value = {
```

```
    for user_name, key in aws_iam_access_key.users_access_keys : user_name => key.secret
```

```
  }
```

```
  sensitive = true
```

```
}
```

```
GNU nano 7.2                                main.tf *
# -----
# Create multiple IAM users from CSV
resource "aws_iam_user" "users" {
  for_each = { for user in local.users : user.user_name => user }

  name      = each.value.user_name
  path      = "/users/"
  force_destroy = true

  tags = {
    DisplayName = each.value.user_name
    CreatedBy   = "Terraform"
  }
}

# Add all users to developers group
resource "aws_iam_user_group_membership" "users_membership" {
  for_each = aws_iam_user.users

  user = each.value.name
  groups = [
    aws_iam_group.developers.name
  ]
}

# Create login profiles for all users
resource "null_resource" "create_login_profiles" {
  for_each = aws_iam_user.users

  triggers = {
    password_hash = sha256(var.iam_password)
    user          = each.value.name
  }

  depends_on = [aws_iam_user.users]

  provisioner "local-exec" {
    command = "${path.module}/create-login-profile.sh ${each.value.name} '${var.iam_password}'"
  }
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

4. Reinitialize Terraform (since we changed the configuration significantly):

terraform init

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/null from the dependency lock file
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/null v3.2.4
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

5. Apply the configuration to create all users:

```
terraform apply -auto-approve -var="iam_password=MySecurePass123!"
```

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform apply -auto-approve -var="iam_password=MySecurePass123!"
```

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

aws_iam_access_key.users_access_keys["Andy"] will be created

```
+ resource "aws_iam_access_key" "users_access_keys" {
  + create_date           = (known after apply)
  + encrypted_secret      = (known after apply)
  + encrypted_ses_smtp_password_v4 = (known after apply)
  + id                    = (known after apply)
  + key_fingerprint       = (known after apply)
  + secret                = (sensitive value)
  + ses_smtp_password_v4  = (sensitive value)
  + status                = "Active"
  + user                  = "Andy"
}
```

aws_iam_access_key.users_access_keys["Angela"] will be created

```
+ resource "aws_iam_access_key" "users_access_keys" {
  + create_date           = (known after apply)
  + encrypted_secret      = (known after apply)
  + encrypted_ses_smtp_password_v4 = (known after apply)
  + id                    = (known after apply)
  + key_fingerprint       = (known after apply)
  + secret                = (sensitive value)
  + ses_smtp_password_v4  = (sensitive value)
  + status                = "Active"
  + user                  = "Angela"
}
```

aws_iam_access_key.users_access_keys["Charles"] will be created

```

    "user_unique_id" = "AIDARMIC7HAZETZPPNAN4"
  }
  "Peter" = {
    "access_key_id" = "AKIARMIC7HAZMW20HLOK"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Peter"
    "user_unique_id" = "AIDARMIC7HAZPO6IMGCL4"
  }
  "Phyllis" = {
    "access_key_id" = "AKIARMIC7HAZK2PJUSMK"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Phyllis"
    "user_unique_id" = "AIDARMIC7HAZIW3RVSB6Z"
  }
  "Robert" = {
    "access_key_id" = "AKIARMIC7HAZF5CQQNQ2"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Robert"
    "user_unique_id" = "AIDARMIC7HAZJKUSXLIHR"
  }
  "Ryan" = {
    "access_key_id" = "AKIARMIC7HAZP6VAJTWP"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Ryan"
    "user_unique_id" = "AIDARMIC7HAZKY5R5B7MH"
  }
  "Stanley" = {
    "access_key_id" = "AKIARMIC7HAZP7KAPWMC"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Stanley"
    "user_unique_id" = "AIDARMIC7HAZN5UZSDY56"
  }
  "Toby" = {
    "access_key_id" = "AKIARMIC7HAZGZACN7GS"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Toby"
    "user_unique_id" = "AIDARMIC7HAZOZHMF6W6F"
  }
}
group_details = {
  "group_arn" = "arn:aws:iam::095032391730:group/groups/developers"
  "group_name" = "developers"
  "unique_id" = "AGPARMIC7HAZA6HZ4KF75"
}
@manahilhabib031 → /workspaces/Lab13 (main) $ |

```

6. Display the outputs:

terraform output

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform output
all_access_key_secrets = <sensitive>
all_users_details = {
  "Andy" = {
    "access_key_id" = "AKIARMIC7HAZE4HAJ7ZB"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Andy"
    "user_unique_id" = "AIDARMIC7HAZFFAARAWLW"
  }
  "Angela" = {
    "access_key_id" = "AKIARMIC7HAZPUMY5VW2"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Angela"
    "user_unique_id" = "AIDARMIC7HAZLSWHTTYFA"
  }
  "Charles" = {
    "access_key_id" = "AKIARMIC7HAZML7KXWBZ"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Charles"
    "user_unique_id" = "AIDARMIC7HAZJKXYKB5FA"
  }
  "Clark" = {
    "access_key_id" = "AKIARMIC7HAZLWJH5NIA"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Clark"
    "user_unique_id" = "AIDARMIC7HAZOH7CXIFVA"
  }
  "Creed" = {
    "access_key_id" = "AKIARMIC7HAZGIDAHYWI"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Creed"
    "user_unique_id" = "AIDARMIC7HAZJ2DXDZO7Y"
  }
  "Darryl" = {
    "access_key_id" = "AKIARMIC7HAZEVSTIYVP"
    "user_arn" = "arn:aws:iam::095032391730:user/users/Darryl"
    "user_unique_id" = "AIDARMIC7HAZNISYANPN6"
  }
  "David" = {
    "access_key_id" = "AKIARMIC7HAZDJQDYQ04"
    "user_arn" = "arn:aws:iam::095032391730:user/users/David"
    "user_unique_id" = "AIDARMIC7HAZML4MZORTA"
  }
}
```

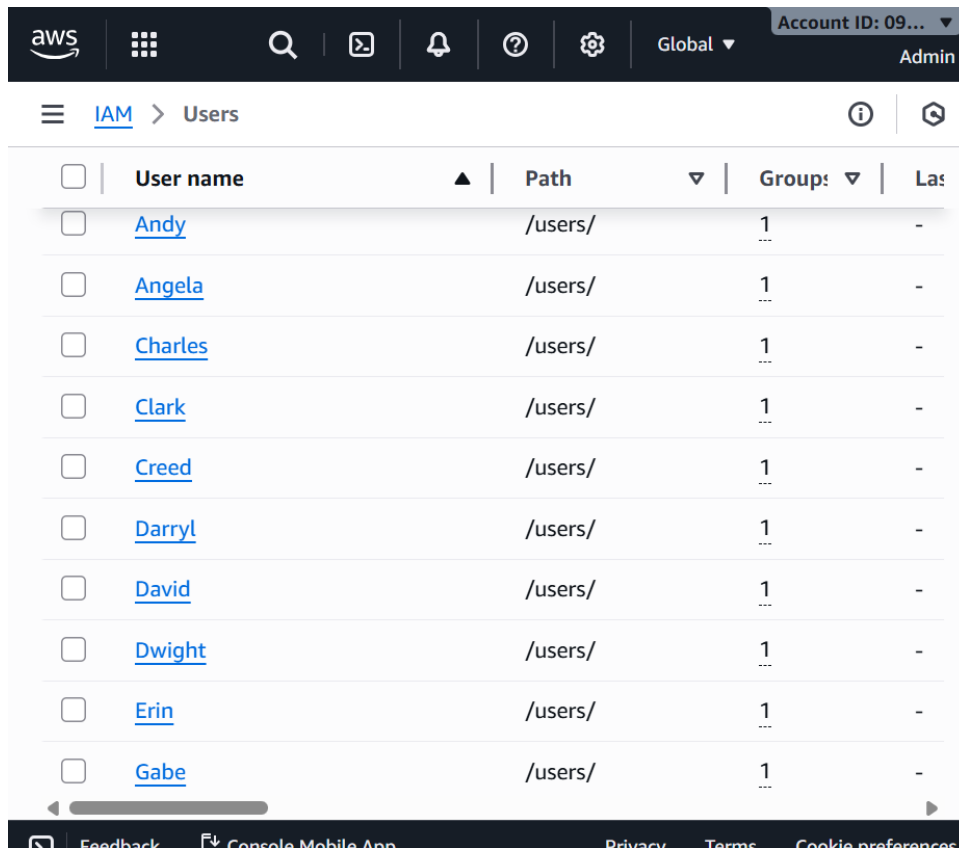
7. View secrets in terraform. tfstate:

```
cat terraform.tfstate | grep -A 5 "all_access_key_secrets"
```

```
@manahilhabib031 → /workspaces/Lab13 (main) $ cat terraform.tfstate | grep -A 5 "a
ll_access_key_secrets"
@manahilhabib031 → /workspaces/Lab13 (main) $ |
```

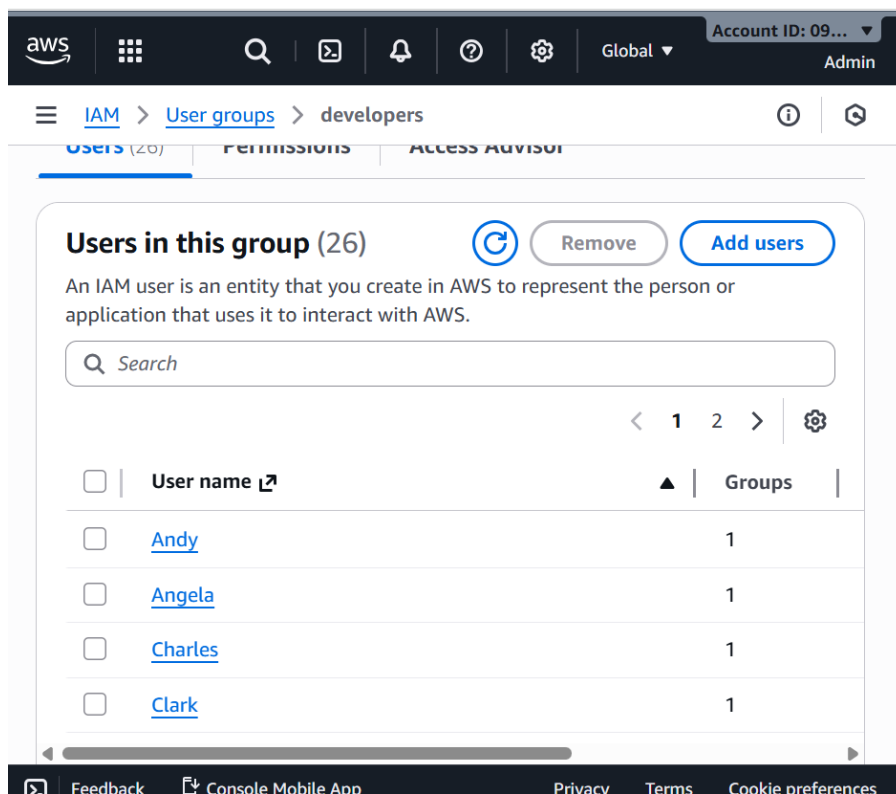
8. Verify all users in AWS Console:

- Navigate to IAM → Users



9. Verify group membership:

- Navigate to IAM → Groups → developers → Users tab



10. Verify one user's access keys:

- Click on any user (e.g., "Michael")
- Go to Security credentials tab

The screenshot shows the AWS IAM console interface. At the top, there's a navigation bar with the AWS logo, a search icon, and a dropdown menu showing 'Global' and 'Account ID: 09...'. Below the navigation bar, the breadcrumb trail reads 'IAM > Users > Andy'. The main content area is titled 'Andy Info' with a 'Delete' button. Under the 'Summary' section, the ARN is 'arn:aws:iam::095032391730:user/users/Andy'. It shows 'Access key 1' as 'AKIARMIC7HAZE4HAJ7ZB - Active' with a note 'Never used. Created today.' and 'Access key 2' with a 'Create access key' link. The 'Console access' section shows 'Enabled without MFA' with a warning icon. The 'Created' date is 'January 11, 2026, 11:06 (UTC+05:00)'. At the bottom, there are tabs for 'Groups (1)', 'Tags (2)', 'Security credentials' (which is selected), and 'Last Accessed'.

11. Check terraform state in S3:

- Navigate to S3 bucket and view the state file

Cleanup

1. Destroy all resources:

terraform destroy -auto-approve

```
@manahilhabib031 → /workspaces/Lab13 (main) $ terraform destroy -auto-approve
aws_iam_group.developers: Refreshing state... [id=developers]
aws_iam_user.users["Phyllis"]: Refreshing state... [id=Phyllis]
aws_iam_user.users["Stanley"]: Refreshing state... [id=Stanley]
aws_iam_user.users["Dwight"]: Refreshing state... [id=Dwight]
aws_iam_user.users["Peter"]: Refreshing state... [id=Peter]
aws_iam_user.users["Kelly"]: Refreshing state... [id=Kelly]
aws_iam_user.users["Meredith"]: Refreshing state... [id=Meredith]
aws_iam_user.users["Jim"]: Refreshing state... [id=Jim]
aws_iam_user.users["Michael"]: Refreshing state... [id=Michael]
aws_iam_user.users["Ryan"]: Refreshing state... [id=Ryan]
aws_iam_user.users["Kevin"]: Refreshing state... [id=Kevin]
aws_iam_user.users["Jan"]: Refreshing state... [id=Jan]
aws_iam_user.users["Charles"]: Refreshing state... [id=Charles]
aws_iam_user.users["Angela"]: Refreshing state... [id=Angela]
aws_iam_user.users["Gabe"]: Refreshing state... [id=Gabe]
aws_iam_user.users["Robert"]: Refreshing state... [id=Robert]
aws_iam_user.users["David"]: Refreshing state... [id=David]
aws_iam_user.users["Darryl"]: Refreshing state... [id=Darryl]
aws_iam_user.users["Holly"]: Refreshing state... [id=Holly]
aws_iam_user.users["Oscar"]: Refreshing state... [id=Oscar]
aws_iam_user.users["Andy"]: Refreshing state... [id=Andy]
aws_iam_user.users["Jo"]: Refreshing state... [id=Jo]
```

```

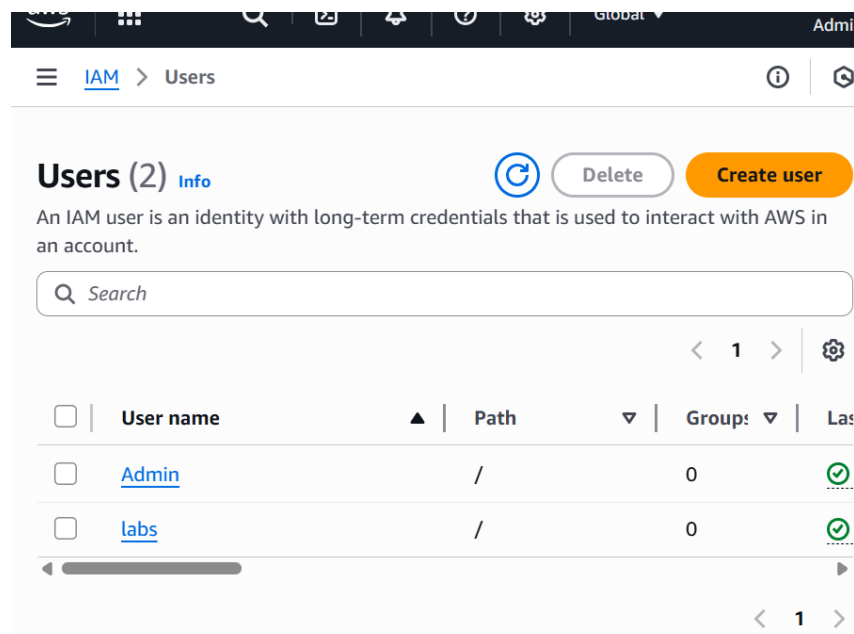
aws_iam_user.users["Angela"]: Destruction complete after 4s
aws_iam_user.users["Kevin"]: Destroying... [id=Kevin]
aws_iam_user.users["Erin"]: Destruction complete after 3s
aws_iam_user.users["Pam"]: Destroying... [id=Pam]
aws_iam_user.users["Jim"]: Destruction complete after 3s
aws_iam_user.users["Clark"]: Destroying... [id=Clark]
aws_iam_user.users["Michael"]: Destruction complete after 3s
aws_iam_user.users["Oscar"]: Destroying... [id=Oscar]
aws_iam_user.users["David"]: Destruction complete after 2s
aws_iam_user.users["Meredith"]: Destroying... [id=Meredith]
aws_iam_user.users["Dwight"]: Destruction complete after 2s
aws_iam_user.users["Darryl"]: Destroying... [id=Darryl]
aws_iam_user.users["Kelly"]: Destruction complete after 2s
aws_iam_user.users["Jan"]: Destroying... [id=Jan]
aws_iam_user.users["Holly"]: Destruction complete after 2s
aws_iam_user.users["Ryan"]: Destroying... [id=Ryan]
aws_iam_user.users["Kevin"]: Destruction complete after 3s
aws_iam_user.users["Stanley"]: Destruction complete after 7s
aws_iam_user.users["Robert"]: Destruction complete after 4s
aws_iam_user.users["Clark"]: Destruction complete after 2s
aws_iam_user.users["Oscar"]: Destruction complete after 3s
aws_iam_user.users["Meredith"]: Destruction complete after 3s
aws_iam_user.users["Darryl"]: Destruction complete after 3s
aws_iam_user.users["Jan"]: Destruction complete after 3s
aws_iam_user.users["Pam"]: Destruction complete after 4s
aws_iam_user.users["Ryan"]: Destruction complete after 3s

Destroy complete! Resources: 107 destroyed.
@manahilhabib031 → /workspaces/Lab13 (main) $ |

```

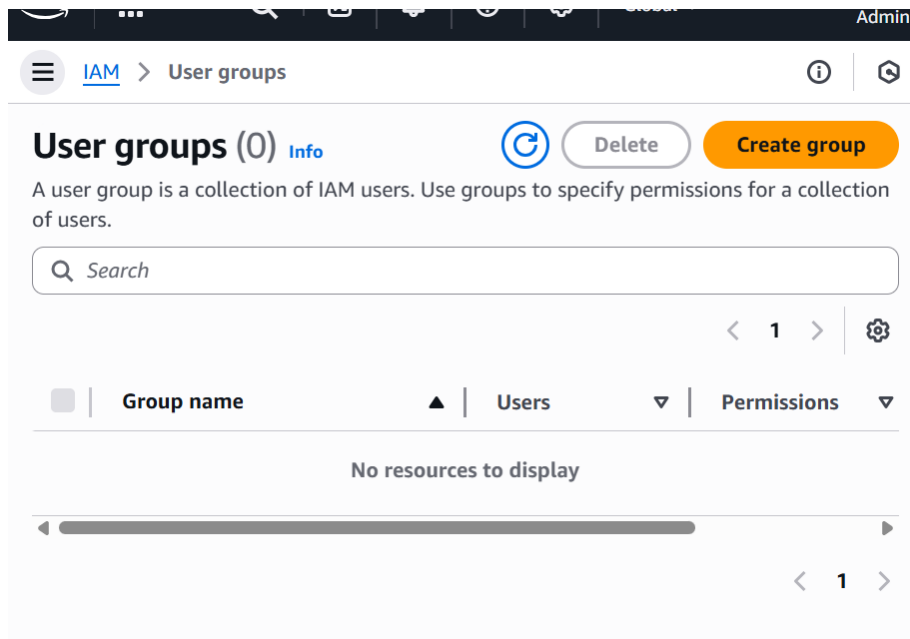
2. Verify users deleted in AWS Console:

- Navigate to IAM → Users



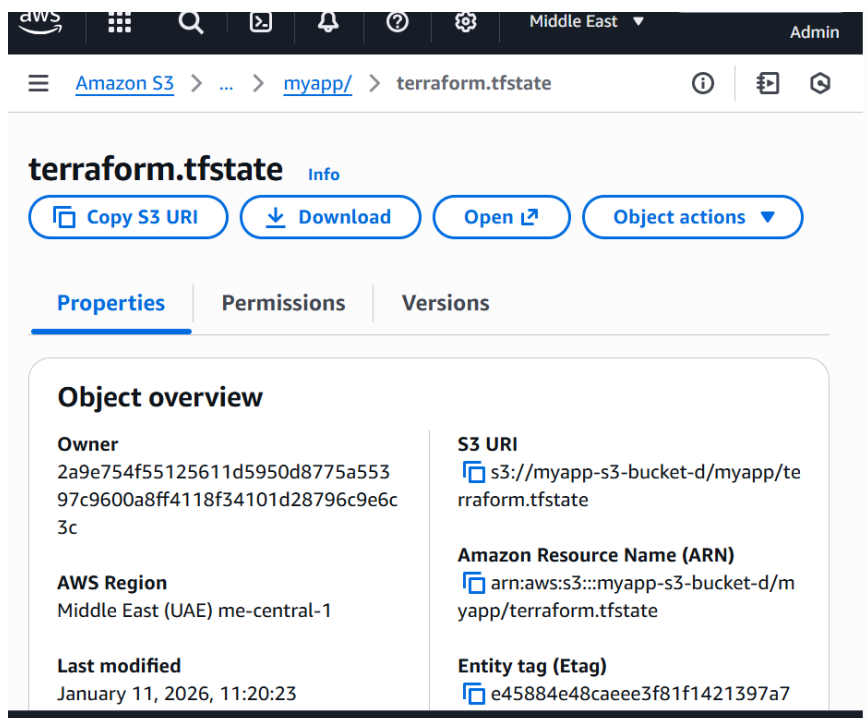
3. Verify group deleted in AWS Console:

- Navigate to IAM → Groups



4. Check S3 state file:

- Navigate to S3 bucket



5. List all project files:

ls -la

```

@manahilhabib031 → /workspaces/Lab13 (main) $ ls -la
total 52
drwxrwxrwx+ 4 codespace root      4096 Jan 11 06:05 .
drwxr-xrwx+ 5 codespace root      4096 Jan 11 03:55 .git
drwxrwxrwx+ 8 codespace root      4096 Jan 11 03:56 .git
drwxr-xr-x+ 3 codespace codespace 4096 Jan 11 06:02 .terraform
-rw-r--r-- 1 codespace codespace 2422 Jan 11 05:10 .terraform.lock.hcl
-rw-rw-rw- 1 codespace root        11 Jan 11 03:55 README.md
-rwxrwxrwx 1 codespace codespace  422 Jan 11 05:06 create-login-profile.sh
-rw-rw-rw- 1 codespace codespace   50 Jan 11 05:53 locals.tf
-rw-rw-rw- 1 codespace codespace 2839 Jan 11 06:05 main.tf
-rw-rw-rw- 1 codespace codespace    0 Jan 11 05:42 terraform.tfstate
-rw-rw-rw- 1 codespace codespace 6339 Jan 11 05:42 terraform.tfstate.backup
-rw-rw-rw- 1 codespace codespace  167 Jan 11 05:52 users.csv
-rw-rw-rw- 1 codespace codespace  163 Jan 11 06:01 variables.tf
@manahilhabib031 → /workspaces/Lab13 (main) $ |

```

6. (Optional) Delete S3 bucket:

- If you want to clean up completely, delete the S3 bucket from AWS Console

 **Successfully emptied bucket "myapp-s3-bucket-d"** 

 View details below. If you want to delete this bucket, use the [delete bucket configuration](#).

 **Successfully deleted bucket "myapp-s3-bucket-d"** 